Back propagation

Tirth S. Pandya 21/10/26

Back propagation, Explained From the Ground Up

Why Dot Products, Transposes, and Weight Updates Work the Way They Do

A Mathematical and Intuitive Walkthrough of Neural Network Training

Tirth S. Pandya

Abstract

Backpropagation is the algorithm by which a neural network learns from its own errors and updates every weight in its architecture. In this paper, the goal is to build up some of the underlying math of backpropagation, relating matrix multiplication, transposition and dot products to the chain rule of calculus. Too often the gradient formulas are thought of as pieces of code. But each formula can be understood as an answer to a question: to what extent should a particular weight in the network be changed to cut down on its error? I hope that each operation in a backward pass will feel like a natural, efficient capture of some underlying mathematical need, and not just a computational convenience.

Introduction

At its most basic, a neural network is just two operations, repeated throughout the network: a linear algebra operation (the weighted sum) and a nonlinearity (e.g. the sigmoid). In order to train the network, the question is asked, for each weight, if this weight is tweaked a little, will the overall error go up or down, and by how much? Efficiently answering that question for each weight in the network is exactly what matrix multiplication, transposition and dot products do. Here the paper works through this connection. It is not rocket science to understand the underlying maths behind this networks.

What a Single Neuron Computes

Each neuron computes z = x1w1 + x2w2 + b. Here, each input is multiplied by a weight (indicating its importance), the results are added up and then shifted by

a bias. In more complex networks, it would be tedious to compute this for each neuron. So it's useful to write it in terms of linear algebra, as shown below.

The Dot Product as the Natural Unit of Computation

$$x.w = x1 * w1 + x2 * w2 + \ldots + xn * wn$$

Term for term, this is exactly the dot product computed by a neuron. More than just an analogy, at its core a neuron computes a dot product. By writing the computation this way, three things are achieved:

1. Speed - Optimized, vectorized low-level routines (BLAS) underlie dot products. They are much faster than loops.
2. Scale - This same expression would work for a neuron with 2 inputs or one with 2,000. The code would not have to change. (But this approach may cause ambiguities in future)
3. Hardware fit - GPUs are built to do dot products in parallel. Because of this, it becomes computationally feasible to train a network on large data.

Matrix Multiplication as Batched Dot Products

A layer is made up of neurons all taking the same inputs. In a matrix multiplication, many dot products are computed at once. Let there be a matrix X of the inputs (each row is an example) and a matrix W of the weights (each column is a neuron).

$$Z = X @ W$$

Each element of Z is the dot product between the inputs of an example and the weights of a neuron. And in one go, this is computed for every example and every neuron. In matrix multiplication, the requirement is that the number of columns of X equals the number of rows of W. That is because in a dot product, the two vectors involved need to be the same length.

Why Transposes Are Structurally Necessary

A transpose swaps the rows and columns of a matrix. In backprop, information must flow backward through the network, from output to input. To do this, the direction of information flow must be reversed. And with it, the shape of the matrices *must be reversed*.

$$\underbrace{X}_{(\text{examples}\times\text{inputs})} @ \underbrace{W}_{(\text{inputs}\times\text{neurons})} = \underbrace{Z}_{(\text{examples}\times\text{neurons})}$$

Backward direction:

$$\text{error}_{\text{input}} = \text{error}_{\text{output}} \cdot W^T$$

W can be viewed as a *one-way function* from input space to output space.

Multiplying by W.T gives a function from output space to input space. Mathematically, this traces the same connections but in the *opposite direction*.

Another use of the transpose is in relating the error in the output to the input values. These have different indices, but through the transpose they can be brought together so that, in effect, each input is matched up with the corresponding error term across all weights.

The Chain Rule: What a Gradient Represents

A gradient tells whether, and by how much, increasing a particular weight would increase or decrease the loss. *If you add more water in your coffee, will it taste the same?*

In backpropagation, the chain rule of derivatives is used to calculate this. It says that if A affects B and B affects C, then A affects C through its effect on B times B's effect on C.

So the gradient of the loss with respect to a weight will be a product of derivatives at each stage of this chain.

weight → z (weighted sum) → a (activation) → ... → loss

That's why the error is multiplied by the derivative of the activation, then by the weights of the next layer, then by the inputs. And this is done, layer by layer, moving backwards through the network.

This gives the derivative of the loss (using mean squared error or anything of your choice) with respect to the prediction.

If the derivative of (a_output - y) 2 is taken with respect to a_output, the result is 2 * (a_output - y). Usually this 2 is included in the learning rate, and so ends up with this term.

Deriving the Backward Pass Step by Step

$$\text{output_error} = a_{\text{output}} - y$$

By the chain rule,

$$\frac{dL}{dz} = \frac{da}{dz} \cdot \frac{dL}{da}$$

Here the second term is the derivative of the activation with respect to its input (i.e. the current sensitivity of the activation to a change in its input). The first term is the error already calculated.

Again, by the chain rule,

$$\frac{dL}{dw} = \frac{dz}{dw} \cdot \frac{dL}{dz}$$

$$\delta_{\text{output}} = \text{output_error} \cdot \sigma'(z_{\text{output}})$$

Since

$$z = a_{\text{hidden}} \cdot w \quad \frac{dz}{dw} = a_{\text{hidden}}$$

So here is the activity of an input, multiplied by the change in output that would be needed. And this is done for all input/neuron pairs at once through the dot product. The transpose of W makes sure that, for each pair, the input and error are aligned. Division by m takes an average of the gradient over the training

examples.

Using the chain rule on the layer before,

$$\frac{dL}{da_{\text{hidden}}} = \frac{dL}{dz_{\text{output}}} \cdot \frac{dz_{\text{output}}}{da_{\text{hidden}}}$$

$$\nabla W_{\text{hidden}\to\text{output}} = \frac{A_{\text{hidden}}^{T} \cdot \delta_{\text{output}}}{m}$$

z_output = a_hidden · W. So the derivative of z_output with respect to a_hidden is W. That means multiplying by W.T pushes error from the output back to each neuron in the hidden layer in proportion to that neuron's effect on the output in the forward pass.

$$\text{hidden_error} = \text{output_delta} \cdot W_{\text{hidden,output}}^{T}$$

$$W_{\text{hidden,output}} \leftarrow W_{\text{hidden,output}} - \eta \cdot \nabla W_{\text{hidden,output}}$$

The gradient points in the direction of fastest increase in loss. To minimize loss, the weight moves in the opposite direction of the gradient.

The learning rate determines the size of this move. If it is too large, the process risks skipping over the minimum; if it is too small, it won't converge.

All of the expressions above could in principle be written as loops over the scalar values of weights and examples. And in early treatments of this, they often are.

In practice, though, in a network, the system considers many examples, many neurons and many layers.

Why Matrix Form Rather Than Explicit Loops

Using matrices, the gradient of loss with respect to each weight and each example can be computed in one go. Under the hood, this relies on fast numerical libraries to do the arithmetic. And the code closely mirrors the calculus behind it.

A neuron computes a weighted sum of its inputs. That's a dot product.

A layer of neurons computes many dot products at once. That's matrix multiplication.

In training, for each weight in each layer, the question is asked: how could it change to reduce error? The chain rule answers this question. And the effect of each layer gets multiplied into the next.

Summary

To compute this, information needs to pass back through the connections used in the forward pass. That means the same weight matrices as before are needed, but transposed, so they have the right shape for this direction.

Each matrix multiplication, dot product and transpose in backprop is a natural, efficient representation of the calculus supporting it. They aren't really a matter of style or implementation.

This was my first try explaining core fundamentals and how maths makes this large machine like GPT, Fable, Argon work.

We unforuntaly do not have their complete architecture, but by connecting this small dots together I hope I was able to make some sense for a topic which is not easy to digest, thank you for your time.

Quick Reference

Concept	**Computes**	**Why It Is Needed**
Dot product	Weighted sum of two vectors	This is what a neuron computes
Matrix multiplication	Many dot products at once	Computes a full layer in one operation
Transpose	Exchanges rows and	Reverses the direction of

	columns	information flow
Sigmoid derivative	Local slope of the activation	Measures current sensitivity to change
Chain rule	Product of layer-wise derivatives	Traces a weight's effect on the final loss
Gradient	Direction of steepest loss increase	Indicates which direction to avoid
Learning rate	Step size	Controls the size of each correction